A Patched-Conic Mission Analysis Tool for Lunar and Interplanetary Design

Modeling approach, validation, utility and limitations

`sfmission` (`sfmission`) v0.1

MECE 4305 Mechanics & Thermodynamics of Propulsion — Rocket Module, Week 10
Columbia University, Fall 2026

9 November 2026

Abstract

This report documents `mission_analysis`, a Python package (with a command-line interface and a Model Context Protocol server) for preliminary design of lunar and interplanetary missions. The tool implements the first four steps of the rocket-module design roadmap—mission definition, required Δv, propellant mass fraction and required I_{sp}, staging—using patched-conic astrodynamics with analytic ephemerides: Hohmann and Lambert transfers, porkchop launch-window maps, planetocentric departure and capture burns, unpowered and powered gravity assists, the Bate–Mueller–White Earth–Moon patched conic, launch Δv with site-rotation credit and loss bands, aerocapture and aerobraking credits, the rocket equation and closed-form optimal multistage sizing, and Δv budgets that resolve directly into propellant masses. An 85-case test suite verifies the implementation against textbook examples, an independent ephemeris, and flown missions: Mars 2020 (heliocentric path length 470.9 Mkm vs. ≈471 Mkm published), Voyager 2's Jupiter flyby, and Apollo 11's translunar coast (68–73 h vs. 73.0 h). We state the modeling assumptions explicitly, quantify their consequences where we can, and identify a factor-$\sqrt{2}$ discrepancy between the straight-line aerobraking formula used in the lecture notes and a gravity-aware form confirmed by direct integration. The limitations section describes what the tool does *not* model and when a higher-fidelity method is required.

Contents

1 Purpose and scope **2**

2 Architecture **2**

3 Modeling approach **3**

4 Verification and validation **6**

5 Utility: what the tool is for **7**

6 Limitations **8**

7 Planned extensions **9**

A Nomenclature **10**

B MCP tools **10**

1 Purpose and scope

The rocket module organizes its lectures and the team project around a ten-step design procedure (Sutton [1]): mission definition → required Δv → propellant mass fraction → required I_{sp} → staging/architecture → stage thrust → single-point thrust-chamber design → $\dot{m}$, V_e → p_c, M_e, A_e, T_c → feed system → iterate. The tool described here covers steps 1–4 and is intended to be:

1. a **lecture instrument** for Week 10 (*Mission Analysis*): live porkchop plots of the Earth–Mars window opening this month, Earth–Moon transfers, and the Δv levers (aerobraking, retro-propulsion, gravity assists);
2. the **reference solution for HW 1** (mission Δv budget and staging) and the first module of each team's project model;
3. a **preliminary-design tool** in its own right, exposed to agents through MCP so that it can be composed with the course's thermochemistry, nozzle and cycle models.

Fidelity class. The tool is a *patched-conic, impulsive-burn* model with *analytic mean-element ephemerides*. It is the appropriate fidelity for sizing a propulsion system and for understanding why a mission's Δv is what it is. It is not a trajectory-design tool: it does not propagate perturbed motion, does not model finite burns, and does not target B-plane conditions. Section 6 makes these boundaries precise.

Interfaces. The same functions are available as a Python API, a CLI and 23 MCP tools returning JSON, so that a design agent can request, e.g., a Mars-orbiter Δv budget on specific dates and hand the injected mass to a staging or engine-sizing tool:

```
from sfmission import porkchop, interplanetary_budget
sfmission porkchop Earth Mars 2026-09-01 2027-02-15 --tof 120 450 --plot pork.png
sfmission-mcp        # MCP server (stdio); tools listed in Appendix B
```

2 Architecture

Figure 1 shows the module dependency graph. Low-level modules work in km, s, rad; high-level functions accept altitudes in km, angles in degrees where the argument name ends in `_deg`, and epochs as `datetime`, ISO-8601 strings or Julian Dates.

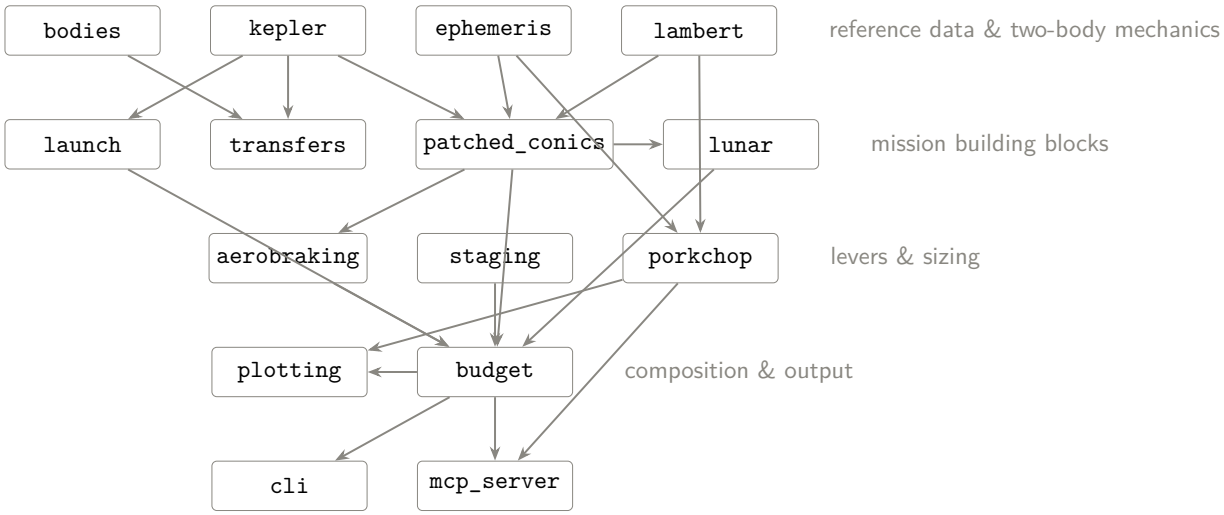


Figure 1: Module dependency graph (arrows point from dependency to dependent). Everything above the `budget` layer is pure functions returning dataclasses; the three front ends only format.

3 Modeling approach

3.1 Reference data

Gravitational parameters are the JPL DE440 values [5, 6] (planet-*system* GMs for the outer planets, which is what a flyby of the system sees); radii are IAU nominal values. Spheres of influence use the Laplace form $r_{SOI} = a\,(m/M)^{2/5}$.

Planetary positions come from JPL's *Keplerian Elements for Approximate Positions of the Major Planets* [4]: six elements and their secular rates, Table 1 valid 1800–2050 (default) and Table 2a/2b valid 3000 BC–3000 AD (used automatically outside Table 1, with the extra M terms for Jupiter–Neptune). Elements are referred to the J2000 ecliptic; velocities are the two-body velocities on the instantaneous mean orbit (the secular element rates contribute $< 1 \times 10^{-6}$ km/s). "Earth" is the Earth–Moon barycenter unless the Moon correction is requested. The Moon is a truncated Meeus series [7] (32 longitude/distance and 20 latitude terms plus the planetary additive terms), precessed from the ecliptic of date to J2000 by the general precession in longitude.

Table 1 compares these models with an independent one (astropy's built-in ERFA `plan94`/`moon98` ephemeris). The agreement is within the accuracy JPL quotes for the tables; for mission design the consequence is a v_∞ error of order 1 m/s at Earth and ∼10 m/s at Saturn—negligible against the Δv margins of a budget.

Table 1: Analytic ephemeris vs. astropy built-in ephemeris, four epochs 2000–2045 (worst case).

Body	angle [″]	rel. distance	velocity [m/s]
Mercury, Venus, Earth(–Moon bary.), Mars	< 35	$< 1.5 \times 10^{-4}$	< 5
Jupiter	< 340	$< 4 \times 10^{-4}$	< 28
Saturn	< 560	$< 1.5 \times 10^{-3}$	< 41
Uranus, Neptune	< 110	$< 1 \times 10^{-3}$	< 16
Moon (geocentric)	< 40	< 40 km	–

3.2 Two-body mechanics

`kepler` provides the vis-viva relation $v^2 = \mu(2/r - 1/a)$, classical elements from and to state vectors (Curtis [2] Algorithms 4.2/4.5), Kepler's equation (Newton from Danby's starting value with a bisection fallback), and universal-variable propagation (Curtis Algorithm 3.4) valid for all conics. Hyperbola relations used by the patched-conic modules are

$$v_p = \sqrt{v_\infty^2 + \frac{2\mu}{r_p}}, \qquad e = 1 + \frac{r_p v_\infty^2}{\mu}, \qquad \delta = 2\arcsin\frac{1}{e}, \qquad \Delta = r_p\sqrt{1 + \frac{2\mu}{r_p v_\infty^2}}, \tag{1}$$

for periapsis speed, eccentricity, asymptotic turning angle and aiming radius.

3.3 Lambert's problem

The heliocentric leg between two ephemeris positions is the single-revolution Lambert solution in the universal-variable formulation [3, 2]: with $A = \sin\Delta\theta\sqrt{r_1 r_2/(1-\cos\Delta\theta)}$ and $y(z) = r_1 + r_2 + A\,(zS(z) - 1)/\sqrt{C(z)}$,

$$F(z) = \left[\frac{y(z)}{C(z)}\right]^{3/2} S(z) + A\sqrt{y(z)} - \sqrt{\mu}\,\Delta t = 0. \tag{2}$$

F is monotonic in z on the single-revolution branch, so the root is bracketed (hyperbolic side $z < 0$ to the $z \to 4\pi^2$ limit, after locating $y = 0$ when $A < 0$) and solved with Brent's method; this never diverges, unlike a bare Newton iteration, and both type-I ($\Delta\theta < \pi$) and type-II ($\Delta\theta > \pi$) transfers fall out of the geometry for prograde motion. A solve costs ≈ 55 µs; a 110×110 porkchop

grid takes about a second. The $\Delta\theta = \pi$ singularity (undefined transfer plane) is reported as an exception and shows up as the familiar ridge in porkchop plots.

3.4 Patched conics

Departure from a circular parking orbit of radius r_p onto a hyperbola with excess speed $v_\infty = |\boldsymbol{v}_{sc} - \boldsymbol{v}_{planet}|$ costs $\Delta v = \sqrt{v_\infty^2 + 2\mu/r_p} - \sqrt{\mu/r_p}$; capture into an orbit with apsides (r_p, r_a) costs $\Delta v = \sqrt{v_\infty^2 + 2\mu/r_p} - \sqrt{\mu(2/r_p - 2/(r_p + r_a))}$. Both are tangential periapsis burns, the optimum for impulsive maneuvers.

An **unpowered flyby** rotates $\boldsymbol{v}_\infty$ by δ about the normal to the $(\boldsymbol{v}_{planet}, \boldsymbol{v}_\infty)$ plane; the heliocentric speed change is $|\Delta v| = 2v_\infty \sin(\delta/2)$. For multi-leg sequences (Earth–Venus–Mars, Earth–Mars–Earth–Jupiter, ...) the incoming and outgoing $\boldsymbol{v}_\infty$ from the adjacent Lambert legs generally differ in both direction and magnitude. The tool uses the standard **powered-flyby** model of the multiple-gravity-assist (MGA) literature [8]: a common periapsis radius r_p is found such that $\frac{1}{2}\delta(v_\infty^{in}, r_p) + \frac{1}{2}\delta(v_\infty^{out}, r_p) = \angle(\boldsymbol{v}_\infty^{in}, \boldsymbol{v}_\infty^{out})$, and the speed mismatch is made up by a tangential burn at periapsis, $\Delta v = \left|\sqrt{(v_\infty^{out})^2 + 2\mu/r_p} - \sqrt{(v_\infty^{in})^2 + 2\mu/r_p}\right|$. If the required turning cannot be achieved above a minimum altitude the flyby is flagged infeasible. This charges the right order of magnitude for "almost-free" assists (Voyager 2 at Jupiter: 0.05 km/s, Section 4) while making infeasible sequences visible.

3.5 Porkchop maps

A grid of departure dates × times of flight is solved leg by leg; each cell records $C_3 = v_\infty^2$ at departure, v_∞ at arrival, the transfer angle, and the departure and capture Δv for the requested parking and capture orbits. The grid optimum, the launch window at a chosen Δv threshold, and the equatorial declination of the launch asymptote (DLA) are derived quantities. Figure 2 is the Earth–Mars window opening in October 2026.

3.6 Earth–Moon patched conic

Two levels are provided. The *Hohmann-to-the-Moon* estimate treats the Moon as a point at the apogee of a transfer ellipse and uses $v_\infty = |v_{Moon} - v_{apogee}|$ for lunar orbit insertion (LOI); it gives the textbook 3.13 km/s TLI and 0.82 km/s LOI. The *Bate–Mueller–White* method [3] (§7.4) patches a geocentric ellipse from the injection state (r_0, v_0, ϕ_0) to the lunar sphere of influence at the point set by the arrival angle λ_1, then a selenocentric hyperbola from $\boldsymbol{v}_2 = \boldsymbol{v}_1 - \boldsymbol{v}_{Moon}$. We implement the geometry with vectors (Moon at $(D, 0)$ moving in $+y$; arrival point $P = \boldsymbol{r}_{Moon} + r_{SOI}(-\cos\lambda_1, \sin\lambda_1)$), so that the sense of motion about the Moon follows from the sign of $h_2 = (\boldsymbol{\rho} \times \boldsymbol{v}_2)_z$ without the sign ambiguities of the scalar relations—which are nevertheless reproduced in the tests. Two additions matter for comparison with flown data: the time spent inside the sphere of influence (hyperbolic Kepler equation from r_{SOI} to perilune, 14 h to 18 h for Apollo-class approaches) is added to the geocentric time of flight, and a design solver returns the λ_1 (for given v_0) or v_0 (for given λ_1) that produces a requested perilune altitude, usually one prograde and one retrograde solution.

3.7 Launch Δv

$$\Delta v_{launch} = \left|\boldsymbol{v}_{orbit} - \boldsymbol{v}_{site}\right| + \Delta v_{grav} + \Delta v_{drag} + \Delta v_{steer}, \qquad \boldsymbol{v}_{site} = \omega_\oplus R \cos\varphi\, \hat{\boldsymbol{e}}_E, \tag{3}$$

with the launch azimuth from $\cos i = \cos\varphi \sin Az$. The vector difference accounts for the launch azimuth needed for the target inclination; due east ($i = \varphi$) it reduces to $v_{circ} - \omega R \cos\varphi$. Losses are carried as a band (default gravity 1.0–1.5, drag 0.05–0.15, steering 0.1–0.4 km/s [1]), so a budget shows $7.726 - 0.409 + (1.15 \ldots 2.05) = 8.47$ km/s to 9.37 km/s for KSC to 300 km. (This also fixes the sign of the Week-9 slide-32 estimate.)

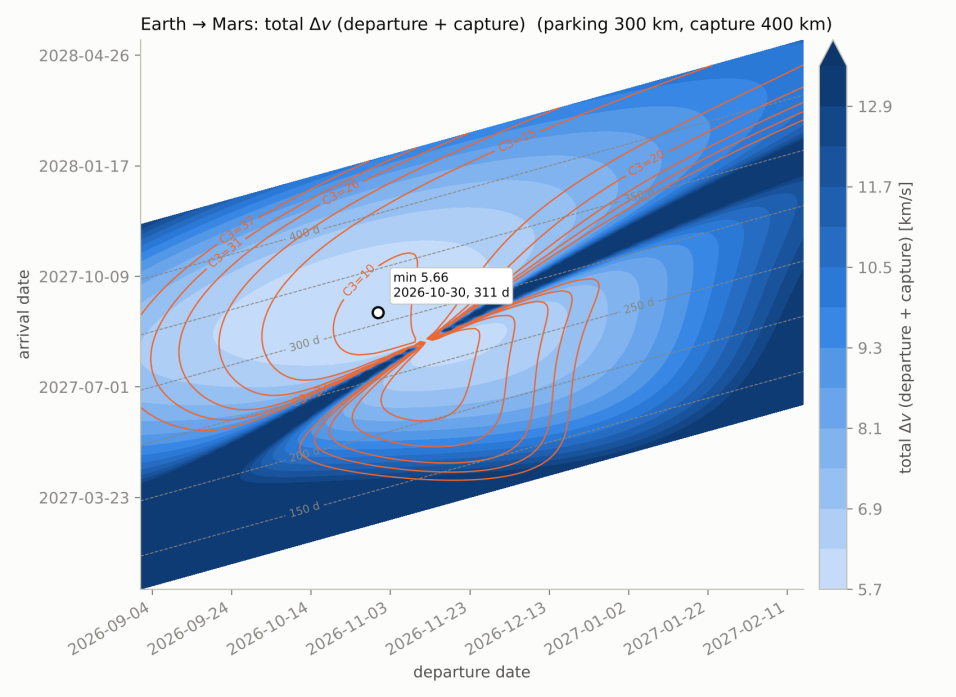


Figure 2: Earth→Mars 2026/27: total Δv (departure from a 300 km parking orbit plus capture into a 400 km Mars orbit); C_3 contours in orange, time of flight dashed. Optimum: depart 2026-10-30, 311 d type II, $C_3 = 9.2\,\mathrm{km^2/s^2}$, $v_\infty^{arr} = 2.57\,\mathrm{km/s}$, $\Delta v = 5.66\,\mathrm{km/s}$; the type-I alternative (2026-11-16, 260 d) costs 6.02 km/s.

3.8 Aerocapture and aerobraking

Aerocapture replaces the propulsive capture burn by one atmospheric pass; the budget credit is the full capture Δv, less the small apoapsis burn that lifts periapsis out of the atmosphere. Entry speed at the interface altitude is reported for heating/g-load assessment, which the tool does not perform.

For aerobraking from a high capture ellipse the tool integrates the apoapsis decay pass by pass. The per-pass speed loss is derived by treating $\rho = \rho_p \exp[-(r - r_p)/H]$ and expanding $r - r_p \approx \frac{1}{2}\ddot{r}_p t^2$ about periapsis:

$$\Delta v_{pass} = \frac{\rho_p V_p^2}{2\beta}\sqrt{\frac{2\pi H}{\ddot{r}_p}}, \qquad \ddot{r}_p = \frac{V_p^2}{r_p} - \frac{\mu}{r_p^2}, \qquad \beta = \frac{m}{C_D A}. \tag{4}$$

Taking $\ddot{r}_p = V_p^2/r_p$ (a straight-line fly-through, gravity neglected) gives the compact form $\rho_p V_p \sqrt{2\pi H r_p}/(2\beta)$ that appears on the Week-11 slide. The two differ by $\sqrt{V_p^2/(V_p^2 - \mu/r_p)}$, which is $\sqrt{2}$ for a parabolic pass and larger for lower apoapses. Direct integration of the planar two-body-plus-drag equations (`aerobraking_pass_dv_numeric`) confirms Eq. (4) to within 0.2% across apoapsis altitudes from 2000 to 160 000 km, while the straight-line form is 30–58% low (Figure 3). The slide formula should be updated accordingly.

3.9 Rocket equation and staging

With Sutton's nomenclature,

$$\zeta = \frac{m_p}{m_0}, \qquad MR = \frac{m_f}{m_0}, \qquad \sigma_k = \frac{m_{s,k}}{m_{s,k} + m_{p,k}}, \qquad c_k = g_0 I_{sp,k}, \qquad r_k = e^{\Delta v_k / c_k}$$

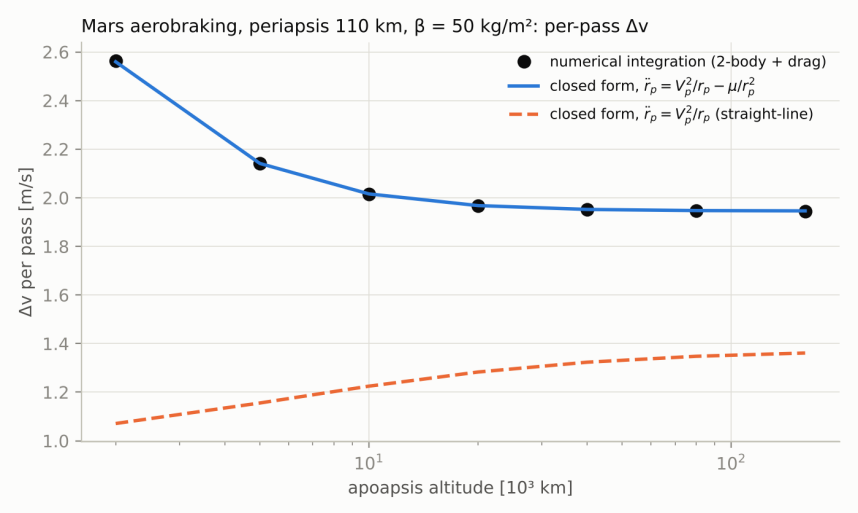


Figure 3: Per-pass Δv at Mars (periapsis 110 km, $\beta = 50\,\mathrm{kg/m^2}$): numerical integration vs. the two closed forms.

(ε is deliberately reserved for nozzle area ratio), the initial-to-payload mass ratio of an N-stage vehicle is

$$\frac{m_0}{m_L} = \prod_{k=1}^{N} \frac{r_k(1-\sigma_k)}{1 - r_k\sigma_k}, \tag{5}$$

and the Δv split minimizing m_0 for fixed $\sum \Delta v_k$ follows from a Lagrange multiplier λ:

$$r_k = \frac{\lambda c_k - 1}{\lambda c_k \sigma_k}, \qquad \sum_k c_k \ln r_k = \Delta v_{tot}, \tag{6}$$

a single scalar root; identical stages recover the equal-split result. Required I_{sp} for a given ζ (roadmap step 3) is $I_{sp} = \Delta v/[g_0 \ln(1/(1-\zeta))]$, and burn sequences are sized backward from the final mass.

3.10 Budgets

A `DeltaVBudget` is an ordered list of legs with nominal/low/high values, a kind (propulsive, aero-assisted, margin) and optionally the I_{sp} of the stage performing the burn. Composers build budgets for dated interplanetary missions (launch band, injection, powered flybys, capture or aerocapture, landing, margin) and for lunar missions (launch, TLI, LOI, Apollo-reference descent/ascent, TEI). Budgets resolve into propellant masses by backward sizing and report propellant fraction and required I_{sp}—the hand-off to the engine design steps.

4 Verification and validation

The test suite (85 cases, ∼1 s) has three layers: unit checks of the mechanics, textbook examples with published numbers, and reconstructions of flown missions from their published dates and states. Table 2 summarizes them.

Two observations from the flown-mission cases are worth recording. First, the Apollo 11 result is as good as the published altitude datum: the two tabulated TLI states (S-IVB cutoff and "TLI" ten seconds later) imply injection energies that differ by $0.04\,\mathrm{km^2/s^2}$, and the lunar time of flight moves by about 5 h per $0.1\,\mathrm{km^2/s^2}$. This sensitivity is physical and is a good lecture point. Second, the Mars 2020 path length agreeing to 0.02% shows that the heliocentric geometry is right; the remaining uncertainty in such reconstructions is in the planetocentric phases, which the patched-conic model only approximates.

Table 2: Validation record.

Case	Result
Lambert, Curtis Example 5.2	$\boldsymbol{v}_1, \boldsymbol{v}_2$ to 1×10^{-4} km/s; propagation closes to 1×10^{-8} km; closure for 20 geometries incl. type II, retrograde and hyperbolic
Elements, Curtis Example 4.3	$h, e, i, \Omega, \omega, \nu$ match
Ephemeris vs. astropy; Mars oppositions 2022/2025/2027	Table 1; Earth/Mars longitudes agree $< 1.5°$ at opposition
Earth→Mars Hohmann (Curtis Ex. 8.4–8.6)	v_∞ 2.943/2.648 km/s, Δv 3.590 km/s from 300 km, phase angle 44.3°, synodic period 779.9 d
LEO→GEO Hohmann; with 28.5° plane change	3.893 km/s; 4.23 km/s with 26.3° at the apogee burn
Mars 2020 (launch 2020-07-30 11:50, entry 2021-02-18 20:37 UTC)	heliocentric path 470.9 Mkm vs. JPL press kit ≈471 Mkm [9]; $C_3 = 14.5\,\mathrm{km^2/s^2}$; $v_\infty^{arr} = 2.55$ km/s consistent with the quoted entry speed
Mars 2020 porkchop	optimum in July 2020 with $C_3 < 15$
Voyager 2 E–J–S dates (1977-08-20, 1979-07-09, 1981-08-25)	Jupiter turn 97°, closest approach 620 000 km (actual ≈570 000 km above the cloud tops), powered correction 0.05 km/s; $C_3 = 105$ (Titan IIIE-Centaur class)
Apollo 11 TLI state [10] (10.832–10.840 km/s at 323–336 km, FPA ≈ 7°) → LOI	TLI-to-perilune 68–73 h vs. 73.0 h actual; LOI 0.89–0.92 km/s vs. 0.891 km/s (into 60×170 nmi)
Lunar Hohmann estimate	TLI 3.135 km/s, LOI 0.82 km/s
Aerobraking pass, Eq. (4) vs. integration	1.952 vs. 1.951 m/s
Staging identities, optimal split, SSTO limit	exact

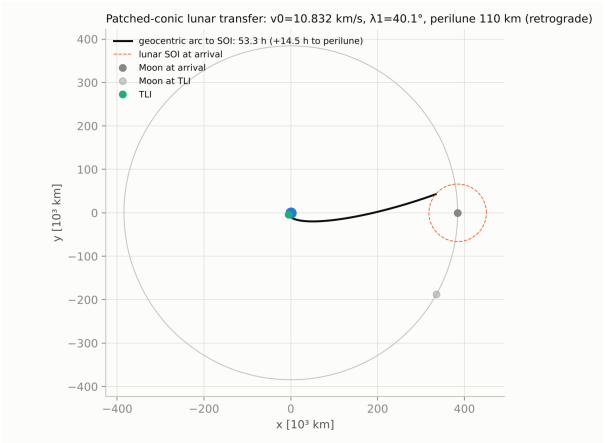
5 Utility: what the tool is for

Lecture. Figure 2 and its companions are generated by `examples/02_mars_porkchop_2026.py` in under a minute, so the Earth–Mars window opening this month can be shown live, including the current phase angle (56° on 11/12 against the 44.3° Hohmann requirement) and the type-I/type-II trade. The lunar module shows why a 3-day transfer costs only 20 m/s more TLI than the 5-day Hohmann estimate but 70 m/s more LOI, and why retrograde lunar orbits fall out of a leading-side approach. The aero and gravity-assist modules quantify the levers (Table 3).

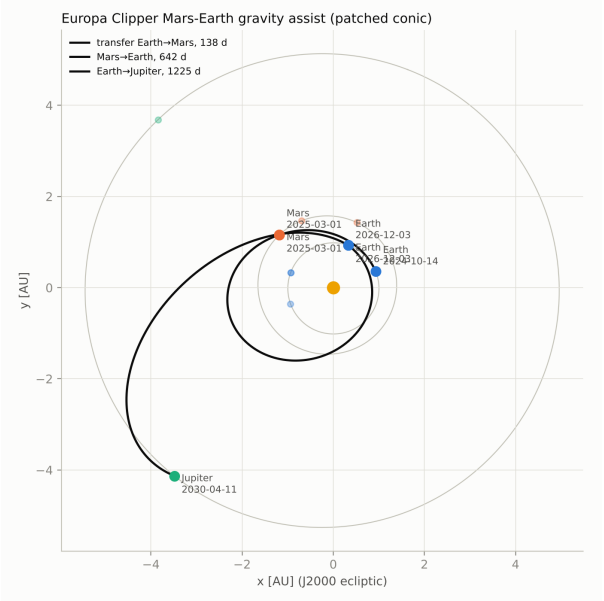
Table 3: Δv levers quantified by the tool (Mars arrival $v_\infty = 2.65$ km/s; Jupiter via Europa Clipper's MEGA dates).

Lever	without	with
Aerocapture into 400 km Mars orbit	2.08 km/s	0.06 km/s (periapsis raise)
Aerobraking 110×40 000 km → circular 400 km, $\beta = 50\,\mathrm{kg/m^2}$	2.08 km/s	0.92 km/s propulsive; 521 passes, 152 d
Mars–Earth gravity assist to Jupiter	$C_3 = 77$, injection 6.30 km/s, 2.7 yr	$C_3 = 41$, injection 4.95 km/s, 5.5 yr
Launch-site rotation credit, KSC due east vs. polar	0.41 km/s	−0.01 km/s

HW 1 and the project model. `examples/06_hw1_reference_solution.py` executes roadmap steps 1–4 for a Mars orbiter launched in the 2026 window: launch band → dated legs → budgets with and without aerocapture (post-launch propulsive Δv 6.39 vs. 4.31 km/s) → required I_{sp} versus propellant fraction (e.g. 541 s at $\zeta = 0.7$ propulsive, 365 s with aerocapture) → spacecraft propellant for a 1000 kg final mass with a 320-s engine (7,653 kg injected) → a two-stage launcher sized for the injected mass (gross 695 t for 12.5 km/s with $\sigma = 0.07/0.10$, $I_{sp} = 300/450$ s). Teams



(a) Apollo-11-like patched-conic transfer (plan view, Earth–Moon plane).



(b) Europa Clipper Mars–Earth gravity assist (patched conic on the published dates).

Figure 4: Geometry outputs used in the Week-10 lecture.

replace the mission, dates and engines and obtain the same chain for their own concept; the `DeltaVBudget` object is the interface to the thermochemistry and engine modules that follow in Weeks 11–12.

Agentic use. Because every function returns a dataclass and the MCP server returns JSON, a supervising design agent can iterate: request a porkchop, pick a date, request the budget, size propellant, call the thermochemistry server for c^* and I_{sp} at a candidate p_c and O/F, and close the loop on injected mass—the workflow the course's project teams are asked to follow by hand.

6 Limitations

The limitations below are consequences of deliberate modeling choices. Each is stated with its expected effect and the remedy when the effect matters.

Patched conics.
Third-body gravity is ignored on each conic; the sphere of influence is a sharp boundary. For Earth–Mars class missions the Δv error of the patched-conic estimate relative to an integrated trajectory is typically a few tens of m/s in the planetocentric burns (a few percent); for the Moon, where the SOI is a large fraction of the Earth–Moon distance, the error in time of flight is hours and in LOI tens of m/s (the Apollo 11 case above shows 0–7%). Remedy: n-body propagation with the patched-conic result as initial guess (planned).

Impulsive burns.
All maneuvers are instantaneous. Finite-burn (gravity) losses for a low-thrust-to-weight injection stage can reach several percent of the burn Δv; launch losses are carried only as a user-set band, not computed from a trajectory. Remedy: the course's ascent-trajectory tool for launch; a finite-burn correction for in-space stages.

Ephemeris accuracy.
Mean elements are good to tens of arcseconds (inner planets) to ∼10 arcmin (Saturn); "Earth" is

the Earth–Moon barycenter by default ($\leq$4700 km offset). Negligible for Δv, unacceptable for targeting. Remedy: DE440 kernels via `jplephem` (planned option).

Circular-orbit assumptions.
Parking and capture orbits are circular or specified by apsides; planetary orbits are circular only in the quick Hohmann estimates (the dated tools use the full elements). The lunar module assumes a circular Moon orbit at 384 400 km and sidereal mean motion; perigee–apogee variation ($\pm$21 000 km) changes TLI by $\sim \pm 10$ m/s and TOF by hours. Remedy: pass the Moon distance on the launch date from `moon_state_geocentric` to the estimate.

Plane and declination constraints.
Heliocentric legs are three-dimensional, but the departure burn assumes the parking orbit can be oriented to contain the asymptote; the equatorial DLA is reported but not constrained, and no plane-change penalty for high-DLA departures or for launch-site/parking-orbit incompatibility is charged.

Lambert.
Single-revolution only; multi-revolution lobes of long-TOF porkchops are absent. Transfers at exactly 180° are undefined (reported as NaN cells).

Powered-flyby idealization.
The shared-periapsis, tangential-burn model is a lower bound on the cost of patching two Lambert legs; real tours use deep-space maneuvers and resonant returns that this model does not optimize. Flyby dates are inputs, not optimized.

Aero-assist.
Aerocapture is charged only its Δv credit; corridor width, peak heating, g-load, heat-shield mass and the guidance problem are not modeled. Aerobraking uses a single-scale-height exponential atmosphere fitted to the 90 km to 150 km band and a constant β; density variability (factor $\sim$2 at Mars) maps directly into pass count and campaign duration. Supply ρ_p from an atmosphere model when it matters.

Staging model.
Structural fraction σ is an input; no engine thrust-to-weight, tank or residual models. The optimal split assumes fixed σ_k independent of stage size.

Reference values in budgets.
Lunar descent/ascent (2.07/1.86 km/s) are Apollo LM reference values, not computed; Mars landing Δv must be supplied.

Not covered.
Low-thrust/electric trajectories (Edelbaum spirals, power-limited rocket equation), launch-vehicle performance curves (C_3 vs. payload), navigation and statistical Δv, radiation/thermal environments, operations constraints.

7 Planned extensions

In priority order for the course: (1) low-thrust legs (Edelbaum Δv, power-limited sizing, trip time) to support HW 3(b) and the electric-propulsion teams; (2) DE440 ephemeris option and a two-body-plus-perturbations propagator to quantify patched-conic error on request; (3) multi-revolution Lambert; (4) launch-vehicle performance curves so a budget resolves to "launcher X can inject m kg at this C_3"; (5) coupling to the course ascent-trajectory optimizer so the launch band becomes a computed value.

A Nomenclature

μ	gravitational parameter GM [km^3/s^2]
v_∞, $C_3 = v_\infty^2$	hyperbolic excess speed, launch energy
δ	flyby turning angle
$\zeta = m_p/m_0$	propellant mass fraction
$MR = m_f/m_0$	mass ratio; $r_k = m_{0,k}/m_{f,k}$ stage mass ratio
$\sigma = m_s/(m_s + m_p)$	stage structural (inert) fraction
$\lambda = m_L/(m_s + m_p)$	payload ratio
λ_1	lunar SOI arrival angle (BMW)
$\beta = m/(C_D A)$	ballistic coefficient [kg/m^2]
H	atmospheric scale height

B MCP tools

`list_bodies`, `planet_state`, `phase_angle`	reference data and geometry
`hohmann_transfer`, `bielliptic_transfer`, `interplanetary_hohmann`	analytic transfers
`lambert_leg`, `porkchop`, `launch_windows`	dated transfers and windows
`departure_dv`, `capture_dv`, `gravity_assist_mission`, `flyby_capability`	patched-conic burns and flybys
`lunar_transfer`, `lunar_mission_budget`	Earth–Moon
`launch_dv`, `aerocapture`, `aerobraking`	launch and aero-assist levers
`stage_vehicle`, `rocket_equation`, `burn_sequence_propellant`	staging and propellant
`mission_budget`, `version`	composition

Configuration for Claude Desktop/Code:

```
{"mcpServers": {"sfmission": {"command": "sfmission-mcp"}}}
```

References

[1] G. P. Sutton and O. Biblarz, *Rocket Propulsion Elements*, 9th ed., Wiley, 2017, Ch. 4.

[2] H. D. Curtis, *Orbital Mechanics for Engineering Students*, 3rd ed., Butterworth-Heinemann, 2014, Ch. 3–8.

[3] R. R. Bate, D. D. Mueller and J. E. White, *Fundamentals of Astrodynamics*, Dover, 1971, Ch. 5, 7, 8.

[4] E. M. Standish, *Keplerian Elements for Approximate Positions of the Major Planets*, JPL Solar System Dynamics, https://ssd.jpl.nasa.gov/planets/approx_pos.html.

[5] R. S. Park et al., "The JPL Planetary and Lunar Ephemerides DE440 and DE441," *Astron. J.* 161:105, 2021.

[6] JPL Solar System Dynamics, *Astrodynamic Parameters*, https://ssd.jpl.nasa.gov/astro_par.html.

[7] J. Meeus, *Astronomical Algorithms*, 2nd ed., Willmann-Bell, 1998, Ch. 47.

[8] D. Izzo, "Global optimization and space pruning for spacecraft trajectory design," in *Spacecraft Trajectory Optimization*, Cambridge, 2010.

[9] NASA/JPL, *Mars 2020 Perseverance Landing Press Kit*, 2021, https://www.jpl.nasa.gov/news/press_kits/mars_2020/landing/mission.

[10] Apollo 11 trajectory tables (from the Apollo 11 Mission Report), https://www.apollomaniacs.com/apollo/flight11_table_e.htm.